

```
import threading
```

```
# =====
```

```
# Класс раздела сайта
```

```
# =====
```

```
class Section:
```

```
    def __init__(self, name: str, code: int):
```

```
        self.name = name
```

```
        self.code = code
```

```
    def __str__(self):
```

```
        return f"Раздел: {self.name}, Код: {self.code}"
```

```
# =====
```

```
# Singleton класс базы данных разделов
```

```
# =====
```

```
class SectionDatabase:
```

```
    _instance = None # статическое поле
```

```
    _lock = threading.Lock() # объект для синхронизации
```

```
    def __init__(self):
```

```
        # защита от создания второго объекта
```

```
        if SectionDatabase._instance is not None:
```

```
            raise Exception("Этот класс является Singleton!")
```

```
        self.sections = [] # список разделов
```

```
# =====
```

```
# Метод получения единственного объекта
```

```
# =====
```

```
@staticmethod
```

```
def initialize():
```

```
    if SectionDatabase._instance is None:
```

```
        with SectionDatabase._lock:
```

```
            if SectionDatabase._instance is None:
```

```
                SectionDatabase._instance = SectionDatabase()
```

```
    return SectionDatabase._instance
```

```
# =====
```

```
# Методы работы с базой
```

```
# =====
```

```
def add_section(self, section: Section):
```

```
    self.sections.append(section)
```

```
def show_sections(self):
```

```
    print("\nСписок разделов:")
```

```

        for section in self.sections:
            print(section)

# =====
# Клиентский код
# =====
if __name__ == "__main__":
    # получаем экземпляр базы данных
    db1 = SectionDatabase.initialize()

    # добавляем разделы
    db1.add_section(Section("Главная", 1))
    db1.add_section(Section("Новости", 2))
    db1.add_section(Section("Контакты", 3))

    db1.show_sections()

    # получаем второй экземпляр
    db2 = SectionDatabase.initialize()

    print("\nПроверка Singleton:")
    print(f"db1 is db2: {db1 is db2}") # True

```

Список разделов:

Раздел: Главная, Код: 1

Раздел: Новости, Код: 2

Раздел: Контакты, Код: 3

Проверка Singleton:

db1 is db2: True

В процессе разработки программных систем часто возникает необходимость ограничить создание объектов определённого класса. Например, при работе с базой данных или конфигурацией системы важно, чтобы существовал только один экземпляр такого класса.

Для решения данной задачи используется шаблон проектирования **«Одиночка» (Singleton)**. В рамках данного практического задания была реализована программа на языке Python, демонстрирующая применение этого паттерна на примере базы данных разделов сайта.

Основная цель работы — изучить механизм создания единственного экземпляра класса и обеспечить глобальный доступ к нему из любой части программы.

Цель работы

Целью данной работы является:

- изучение паттерна «Singleton»;
- реализация класса с единственным экземпляром;
- обеспечение глобальной точки доступа к объекту;
- закрепление навыков работы с классами и статическими полями;
- понимание принципов ограничения создания объектов.

Теоретическая часть

Паттерн **Singleton (Одиночка)** — это порождающий шаблон проектирования, который гарантирует, что у класса существует только один экземпляр, а также предоставляет глобальную точку доступа к этому экземпляру.

Суть данного паттерна заключается в том, что:

- класс сам контролирует создание своих объектов;
- при попытке создать новый объект возвращается уже существующий;
- доступ к объекту осуществляется через специальный метод.

Преимущества паттерна:

- контроль над количеством экземпляров;
- удобный доступ к общим ресурсам;
- экономия памяти;
- централизованное управление данными.

Недостатки:

- нарушение принципа единственной ответственности;
- сложности в многопоточной среде;
- усложнение тестирования.

Постановка задачи

Согласно заданию, необходимо реализовать систему, в которой:

- существует сайт с разделами;
- каждый раздел имеет:
 - название;
 - уникальный код;
- требуется создать класс базы данных `SectionDatabase`, в котором будут храниться разделы;
- класс базы данных должен быть реализован по паттерну Singleton, то есть иметь только один экземпляр;

- необходимо обеспечить потокобезопасность;
- реализовать методы для добавления и вывода разделов.

Объект синхронизации `_lock`

Используется для обеспечения потокобезопасности. Благодаря этому механизму несколько потоков не смогут одновременно создать несколько экземпляров класса.

Метод `initialize()`

Данный метод является точкой доступа к объекту. Его задача:

- проверить, создан ли уже объект;
- если нет — создать новый;
- если да — вернуть существующий.

Также внутри метода используется блокировка (`lock`) для безопасной работы в многопоточной среде.

3. Клиентский код

В основной части программы:

1. Получается экземпляр базы данных через метод `initialize()`;
2. Добавляются разделы сайта;
3. Выводится список разделов;
4. Повторно вызывается `initialize()`;
5. Проверяется, что оба объекта являются одним и тем же экземпляром.

Заключение

В ходе выполнения практической работы был изучен и реализован паттерн «Одиночка». Данный шаблон позволяет контролировать создание объектов и обеспечивает единую точку доступа к ним.

Практическая реализация показала, что Singleton особенно полезен при работе с общими ресурсами, такими как базы данных, конфигурации и менеджеры состояния.

Полученные знания могут быть применены при разработке реальных приложений, где важно обеспечить единый доступ к данным и избежать создания дублирующих объектов.